

In Class Assignment Two

In the following assignment you will be asked to fill in python code and derivations for a number of different problems. Please read all instructions carefully and turn in the rendered notebook (or HTML of the rendered notebook) before the end of class (or right after class). The initial portion of this notebook is given before class and the remainder is given during class. Please answer the initial questions before class, to the best of your ability. Once class has started you may rework your answers as a team for the initial part of the assignment.

Contents

- Loading the Data
 - Defining a Feedforward Network in Python
 - Available during in class assignment:
 - Back Propagation in Python
 - Visualizing Back Propagation
-

[Back to Top](#)

Loading the Data

Please run the following code to read in the “digits” dataset from sklearn’s data loading module. This data contains hand written digits for the characters 0-9.

This will load the data into the variable `ds`. `ds` is a **bunch** object with fields like `ds.data` and `ds.target`. The field `ds.data` is a numpy matrix of the continuous features in the dataset. **The object is not a pandas dataframe. It is a numpy matrix.** Each row is a set of observed instances, each column is a different feature. It also has a field called `ds.target` that is an integer value we are trying to predict (i.e., a specific integer represents a specific person). Each entry in `ds.target` is a label for each row of the `ds.data` matrix.

```
library(reticulate)
py_require("matplotlib")
py_require("numpy")
py_require("pandas")
py_require("scikit-learn")
py_require("scipy")

# lets load up the handwritten digit dataset
from sklearn.datasets import load_digits
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
import numpy as np

ds = load_digits()
X = ds.data/16.0-0.5
y = ds.target

print(X.shape)

## (1797, 64)

print(y.shape)

## (1797,)
```

```

print(np.min(X),np.max(X))

## -0.5 0.5
print(np.unique(y))

## [0 1 2 3 4 5 6 7 8 9]
# reshape and print a few of the images in the digits dataset
import matplotlib.pyplot as plt

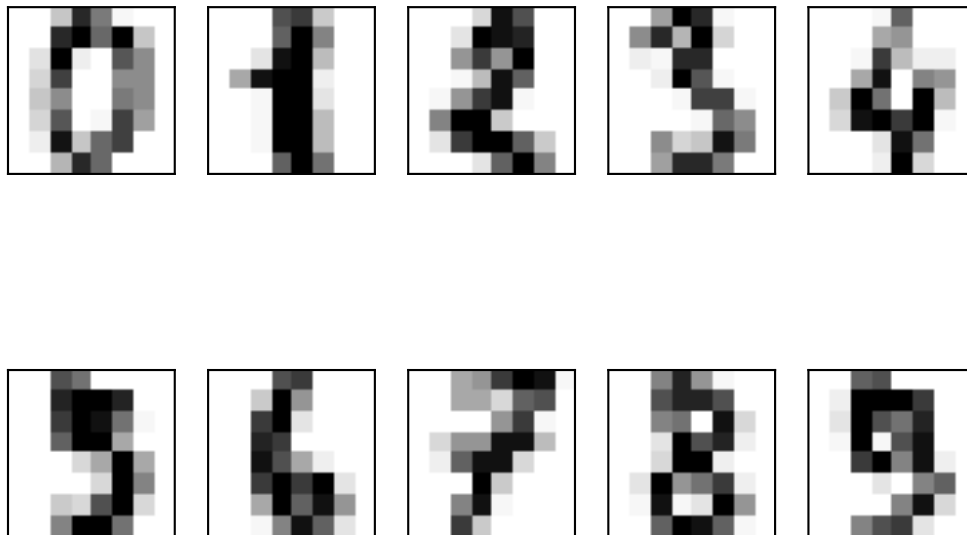
fig, ax = plt.subplots(nrows=2, ncols=5, sharex=True, sharey=True,)
ax = ax.flatten()
for i in range(10):
    img = X[i].reshape(8, 8)
    ax[i].imshow(img, cmap='Greys', interpolation='nearest')

ax[0].set_xticks([])

## []
ax[0].set_yticks([])

## []
plt.show()

```



Question 1: For the digits dataset, what does each column in $\mathbf{X}$ represent? What does each row in $\mathbf{X}$ represent? What does each value in $\mathbf{X}$ represent? What does each unique value of the target, y represent?

- Columns in X represent the features, 64 pixels (the images are 8×8)
- Rows in X represent samples (there are 1,797 images)
- Values in X represent pixel data, in the range 0 to 16, transformed to -0.5 to 0.5
- Unique values of y are the actual digits 0 to 9

Question 2: For the digits dataset, we want to train a neural network with one hidden layer. The hidden layer will have 30 neurons. What will be the size of the matrices in each layer? That is, what is the size of $\mathbf{W}^{(1)}$ and what is the size of $\mathbf{W}^{(2)}$?

- Size of $W^{(1)}$ is 30×64
 - Size of $W^{(2)}$ is 10×30
-

Back to Top

We have an input x that gets multiplied by W and is passed through a non-linearity / activation function ϕ .

There are also intermediate variables a and z which are the results of performing some multiplications, denoted by the layer which they are located using superscript parentheses.

$$\vec{z}^{(1)} = W\vec{a}^{(1)}, \text{ where } W = \begin{bmatrix} {}_1W^{(1)} \\ {}_2W^{(1)} \\ \vdots \\ {}_{st}W^{(1)} \end{bmatrix}$$

When we want to denote the output of a in the l layer from the k instance, we denote it as $[a^{(l)}]^{(k)}$.

$$\text{In matrix form, } X = \begin{bmatrix} x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(m)} \end{bmatrix}, A^{(1)} = x^\top = \begin{bmatrix} [a^{(1)}]^{(1)} & [a^{(1)}]^{(2)} & \dots & [a^{(1)}]^{(M)} \end{bmatrix} \text{ such that each row of } X \text{ is a}$$

separate instance, and each column is a feature. Thus, $Z^{(1)} = W^{(1)}A^{(1)}$, $A^{(2)} = \phi(Z^{(1)})$

In general, $a^{(l+1)} = \phi(W^{(l)}a^{(l)}) = \phi(W^{(l)}\phi(W^{(l-1)}a^{(l-1)}))$

Let $S^{(0)} = N$, and S for the final layer must be the number of predicted classes. Then, $a^{(l)}$ has size $S^{(l-1)} - 1$, $W^{(l)}$ has $S^{(l)} \times S^{(l-1)}$, $Z^{(l)}$ has $S^{(l)} \times 1$.

If our objective function is the mean squared error $J(w) = \sum_{k=1}^M |y^{(k)} - [a^{(L)}]^{(k)}|^2$, we use the chain rule to compute the derivative to get the gradient of the objective function with respect to each $W^{(l)}$.

That is, $W^{(l)} + = \eta \frac{\delta J(w)}{\delta W^{(l)}}$ and likewise for each $W_{ij}^{(l)}$.

Define the sensitivity $v_i^{(l)} = \frac{\delta J}{\delta z_i^{(l)}}$.

Thus, $W^{(l)} + = \eta \sum_{k=1}^m [v^{(l)}]^{(k)} [a^{(l)\top}]^{(k)}$

$$\text{Let } V^{(l)} = \begin{bmatrix} [v^{(l)}]^{(1)} & [v^{(l)}]^{(2)} & \dots & [v^{(l)}]^{(M)} \end{bmatrix}, [A^{(l)}]^\top = \begin{bmatrix} [a^{(l)\top}]^{(1)} \\ [a^{(l)\top}]^{(2)} \\ \vdots \\ [a^{(l)\top}]^{(M)} \end{bmatrix}$$

Then, $W^{(l)+} = \eta V^{(l)} [A^{(l)}]^\top$.

$$V^{(l)} = \frac{\delta J}{\delta \vec{z}^{(l)}} = \left[\frac{\delta \vec{z}^{(l+1)}}{\delta \vec{z}^{(l)}} \right]^\top \frac{\delta J}{\delta \vec{z}^{(l+1)}}.$$

$D = \left[\frac{\delta \vec{z}^{(l+1)}}{\delta \vec{z}^{(l)}} \right]^\top$ is a special matrix called the Jacobian.

Suppose that ϕ is a sigmoid. Then $\dot{\Phi}^{(l+1)} = \text{DIAG}(\vec{\phi}^{(l+1)})$ and $D = W^{(l+1)} \dot{\Phi}^{(l+1)}$.

$$\therefore \vec{v}^{(l)} = D^\top \frac{\delta J}{\delta \vec{z}^{(l+1)}} = \dot{\Phi}^{(l+1)} \left[W^{(l+1)} \right]^\top \vec{v}^{(l+1)}$$

This recurrence relation is called back-propagation.

Let us define the Hadamard (elementwise) product $\odot$.

Then, $\vec{v}^{(l)} = \vec{\Phi}^{(l+1)} \odot \left[W^{(l+1)} \right]^\top \vec{v}^{(l+1)}$.

In matrix form, $V^{(l)} = \vec{\Phi}^{(l+1)} \odot \left[W^{(l+1)} \right]^\top V^{(l+1)}$, where each is of size $S^{(l)} \times M, S^{(l)} \times S^{(l+1)}, S^{(l+1)} \times M$ respectively.

$$\therefore V^{(l)} = A^{(l+1)} \odot (1 - A^{(l+1)}) \odot \left[W^{(l+1)} \right]^\top V^{(l+1)}$$

At the last layer, we define $V^{(L-1)} = -2(Y - A^{(L)}) \odot A^{(L)} \odot (1 - A^{(L)})$

Defining a Feedforward Network

Below we will setup the functions for use in a feedforward neural network **with two layers**. Take a quick look at the functions defined. There are a number of convenience functions including: - a function for the sigmoid calculation - a function to one hot encode the output - a function to add ones (for intercept/bias) terms on the row or column - an initialization function for initializing the weights

A few functions are not yet implemented including: - a `fit` function - a `get_gradient` function

```
# Example adapted from
# https://github.com/rasbt/python-machine-learning-book/blob/master/code/ch12/ch12.ipynb
# Original Author: Sebastian Raschka

# This is the optional book we use in the course, excellent intuitions and
# straightforward programming examples
# please note, however, that this code has been manipulated
# to reflect our assumptions and notation.

import numpy as np
from scipy.special import expit
import sys
import pandas as pd

# start with a simple base classifier, which can't be fit or predicted
# it only has internal classes to be used by classes that will subclass it
class TwoLayerPerceptronBase(object):
    def __init__(self, n_hidden=30,
                  C=0.0, epochs=500, eta=0.001, random_state=None):
        np.random.seed(random_state)
```

```

self.n_hidden = n_hidden
self.l2_C = C
self.epochs = epochs
self.eta = eta

@staticmethod
def _encode_labels(y):
    """Encode labels into one-hot representation"""
    onehot = pd.get_dummies(y).values.T

    return onehot

def _initialize_weights(self):
    """Initialize weights with small random numbers."""
    W1_num_elems = (self.n_features_ + 1)*self.n_hidden
    W1 = np.random.uniform(-1.0, 1.0, size=W1_num_elems)
    W1 = W1.reshape(self.n_hidden, self.n_features_ + 1) # reshape to be W

    W2_num_elems = (self.n_hidden + 1)*self.n_output_
    W2 = np.random.uniform(-1.0, 1.0, size=W2_num_elems)
    W2 = W2.reshape(self.n_output_, self.n_hidden + 1)
    return W1, W2

@staticmethod
def _sigmoid(z):
    """Use scipy.special.expit to avoid overflow"""
    # 1.0 / (1.0 + np.exp(-z))
    return expit(z)

@staticmethod
def _add_bias_unit(X, how='column'):
    """Add bias unit (column or row of 1s) to array at index 0"""
    if how == 'column':
        ones = np.ones((X.shape[0], 1))
        X_new = np.hstack((ones, X))
    elif how == 'row':
        ones = np.ones((1, X.shape[1]))
        X_new = np.vstack((ones, X))
    return X_new

def _feedforward(self, X, W1, W2):
    """Compute feedforward step
    -----
    X : Input layer with original features.
    W1: Weight matrix for input layer -> hidden layer.
    W2: Weight matrix for hidden layer -> output layer.
    -----
    a1-a3 : activations into layer (or output layer)
    z1-z2 : layer inputs

    """
    A1 = self._add_bias_unit(X.T, how='row')
    Z1 = W1 @ A1

```

```

A2 = self._sigmoid(Z1)

A2 = self._add_bias_unit(A2, how='row')
Z2 = W2 @ A2
A3 = self._sigmoid(Z2)
return A1, Z1, A2, Z2, A3

def predict(self, X):
    """Predict class labels"""
    _, _, _, _, A3 = self._feedforward(X, self.W1, self.W2)
    y_pred = np.argmax(A3, axis=0)
    return y_pred

```

Recall from the videos that all instances in $\mathbf{X}$ can be fed into the network with a single matrix multiplication operation for each intermediate vector, $a^{(l)}$ and $z^{(l)}$. When we feed all the instances, $\mathbf{X}$, the intermediate vectors, $a^{(l)}$ and $z^{(l)}$ get stacked together to form matrices, $\mathbf{A}^{(l)}$ and $\mathbf{Z}^{(l)}$. This is already done for you in the `_feedforward` function defined above.

Question 3: For the digits dataset we are using and a network with 30 neurons in the hidden layer, what are the sizes of:

- **Part A:** the intermediate vectors, $a^{(1)}$ and $a^{(2)}$
- **Part B:** the intermediate vectors, $z^{(1)}$ and $z^{(2)}$
- **Part C:** the intermediate matrices, $\mathbf{A}^{(1)}$ and $\mathbf{A}^{(2)}$
- **Part D:** the intermediate matrices, $\mathbf{Z}^{(1)}$ and $\mathbf{Z}^{(2)}$

A. Size of $a^{(1)}$ is 64×1

Size of $a^{(2)}$ is 30×1

B. Size of $z^{(1)}$ is 30×1

Size of $z^{(2)}$ is 10×1

C. $M = 1797$

Size of $\mathbf{A}^{(1)}$ is 64×1797

Size of $\mathbf{A}^{(2)}$ is 30×1797

D. Size of $\mathbf{Z}^{(1)}$ is 30×1797

Size of $\mathbf{Z}^{(2)}$ is 10×1797
